Robotic Manipulation

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2026
Last modified 2026-10-11.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Fall 2026 semester.

Previous Chapter Table of contents Next Chapter

Chapter 11
Reinforcement Learning

These days, there is a lot of excitement around reinforcement learning (RL), and a lot of literature available. The scope of what one might consider to be a reinforcement learning algorithm has also broaden significantly. The classic (and now updated) and still best introduction to RL is the book by Sutton and Barto [1] . For a more theoretical view, try [2, 3]. There are some great online courses, as well: Stanford CS234, Berkeley CS285, DeepMind x UCL.

My goal for this chapter is to provide enough of the fundamentals to get us all on the same page, but to focus primarily on the RL ideas (and examples) that are particularly relevant for manipulation. And manipulation is a great playground for RL, due to the need for rich perception, for operating in diverse environments, and potentially with rich contact mechanics. Many of the core applied research results have been motivated by and demonstrated on manipulation examples!

11.1 RL Software

There are now a huge variety of RL toolboxes available online, with widely varying levels of quality and sophistication. But there is one standard that has clearly won out as the default interface with which one should wrap up their simulator in order to connect to a variety of RL algorithms: the Gymnasium (which is often still called "Gym" for short).

It's worth taking a minute to appreciate the difference in the Gymnasium Environments (`gymnasium.Env`) interface and the Drake System interface; I think it is very telling. My goal (in Drake), is to present you with a rich and beautiful interface to express and optimize dynamical systems, to expose and exploit all possible structure

in the governing equations. The goal in Gym is to expose the absolute minimal details, so that it's possible to easily wrap every possible system under the same common interface (it doesn't matter if it's a robot, an Atari game, or even a compiler). Almost by definition, you can wrap any Drake system as a Gym environment.

Example 11.1 (Using a Drake simulation as an Gym environment)

A Gym environment is an incredibly simple wrapper around simulators which offers a very basic interface, most notably consisting of `reset()`, `step()`, `render()`. The `step()` method returns the current observations and the one-step reward (as well as some additional termination conditions).

You can wrap any Drake simulation in an OpenAI gym environment, using

```
from pydrake.gym import DrakeGymEnv
```

The `DrakeGymEnv` constructor takes a `Simulator` as well as an input port to associate with the actions, an output port to associate with the observations, etc. For the reward, you can implement it as a simple function of the `Simulator Context`, or as another output port.

`DrakeGymEnv` is built around a `Simulator` (not just a `System`) or a function that produces a random `Simulator` because you might want to control the integrator parameters or have each rollout contain the same robot in a different environment, with potentially different numbers of objects. This would mean that the underlying `System` might have a different number of states / ports. The notion of a function that can produce simulators, referred to as a `SimulatorFactory`, is core to the stochastic system modeling framework in Drake.

You can also use any Gym environment in the Drake ecosystem; you just won't be able to apply some of the more advanced algorithms that Drake provides. Of course, I think that you should use Drake for your work in RL, too (many people do), because it provides a the rich library of dynamical systems that are rigorously authored and tested, including a great physics engine for dealing with contact, and leaves open the option to put RL approaches head-to-head against more model-based alternatives. I admit I might be a little biased. At any rate, that's the approach we will take in these notes.

Some people might argue that the more thoughtfully you model your system, the more assumptions you have baked in, making yourself susceptible to "sim2real" gaps; but I think that's simply not the case. Thoughtful modeling includes making uncertainty models that can account for as narrow or broad of a class of systems as we aim to explore; good things happen when we can make the uncertainty models themselves *structured*. I think one of the most fundamental challenges waiting for us at the intersection of reinforcement learning and control is a deeper understanding of

the class of models that is rich enough to describe the diversity and richness of problems we are exploring in manipulation (e.g. with RGB cameras as inputs) while providing somewhat more structure that we can exploit with stronger algorithms. Importantly, these models should continually expand and improve with data.

Gymnasium provides an interface for RL environments, but doesn't provide the implementation of the actual RL algorithms. There are a large number of popular repositories for the algorithms, too. As of this writing, I would recommend Stable Baselines 3: it provides a very nice and thoughtfully-documented set of implementations in PyTorch.

One other class of algorithms that is very relevant to RL but not specifically designed for RL is algorithms for black-box optimization. I quite like Nevergrad, and will also use that here.

11.2 Policy-gradient methods

11.2.1 Black-box optimization

Example 11.2 (Black-box optimization.)

Open in Colab

11.2.2 Stochastic optimal control

11.2.3 Using gradients of the policy, but not the environment

You can find more details on the derivation and some basic analysis of these algorithms here.

11.2.4 REINFORCE, PPO, TRPO

Example 11.3 (Flipping up the box.)

Open in Colab

11.2.5 Control for manipulation should be easy

This is a great time for theoretical RL + controls, with experts from controls embracing new techniques and insights from machine learning, and vice versa. As a simple example, we've increasingly come to understand that, even though the cost landscape for many classical control problems (like the linear quadratic regulator) is not convex in the typical policy parameters, we now understand that gradient descent still works for these problems (there are no local minima), and the class of problems/parameterizations for which we can make statements like this is growing rapidly.

11.3 Value-based methods

11.4 Model-based RL

11.5 Exercises

Exercise 11.1 (Stochastic Optimization)

For this exercise, you will implement a stochastic optimization scheme that does not require exact analytical gradients. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Implement gradient descent with exact analytical gradients.
b. Implement Gaussian zeroth-order gradient estimates with the correct variance scaling, and compare smoothing with sampling noise.
c. Prove that the estimators target the Gaussian-smoothed loss and that baselines leave their expectation unchanged.
d. Implement stochastic gradient descent with baselines.

Exercise 11.2 (REINFORCE)

For this exercise, you will implement the vanilla REINFORCE algorithm on a box pushing task. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Implement the policy loss function.
b. Implement the value loss function.
c. Implement the advantage function.

Exercise 11.3 (Analyzing Box Flipping with RL)

In this exercise, you will analyze the behavior of a PPO policy trained to flip over a box. Like REINFORCE, PPO is a policy-gradient method that directly optimizes the policy parameters to maximize the value function. In order to have an easier problem to analyze, we'll use the box flipup example from Chapter 8. Our robot will be a simple point finger and the goal will be to flip over the box. You can find the code used to train the policy here.

a. Take a look at the code used to generate the environment. Let θ denote the angle of the box from the vertical, ω denote the angular velocity of the box, q_f denote the observed position of the finger, v_f denote the velocity of the finger, and u_f denote the commanded position of the finger. What is the reward function used here to train the policy? Write it down mathematically (use the modulo operator to handle the wrap-around of the angle). What do the individual terms in the reward function represent? Why do they make sense?

b. Although we will not go into the exact details of how PPO works here, it works quite similarly to REINFORCE but using both (i) a learned value function to reduce variance, and (ii) an approximate objective, along with a trust-region constraint by clipping the per-sample loss to ensure that the policy is not updated too much at each step. Briefly explain why you think that (a) PPO might be more stable and sample efficient than REINFORCE, and (b) how you might expect PPO to perform on the box flipping task if the clipping limits are set to be too small or too large.
c. We've trained a PPO-based policy to flip the box for 3,000,000 steps (see here for videos of the policy in action at each of the timesteps). How does the policy perform as the number of steps increases? Write qualitatively how the policy changes over time and which parts of the reward function are having the greatest effect at each step.

Notice how much time it takes to train a working policy, even for a simple manipulation problem like the 2D box flipping example with a point finger and a dense reward. Harder problems in manipulation (such as pick and place) can become extremely challenging to train na 茂 vely with Reinforcement Learning, especially with sparse rewards such as in typical pick and place tasks where you only receive a reward when the object has been picked or placed in the right location. On the other hand, reinforcement learning can work well in contact-rich settings (as in the box flipping example); see RL solving a rubik's cube with one hand for an example of RL being used to solve a contact-rich manipulation task (note this also depended heavily on things like domain randomization, curriculum learning, large scale compute, etc.). The story in locomotion, on the other hand, seems to be quite different, perhaps because it is easier to design dense rewards and to automate resets in simulation.

References

1. Richard S. Sutton and Andrew G. Barto, "Reinforcement Learning: An Introduction", MIT Press , 2018.

2. Alekh Agarwal and Nan Jiang and Sham M. Kakade and Wen Sun, "Reinforcement Learning: Theory and Algorithms", Online Draft , 2020.

3. Csaba Szepesvari, "Algorithms for Reinforcement Learning", Morgan and Claypool Publishers , 2010.

© Russ Tedrake, 2026